

Implementation Summary

Scenario: HR Recruitment Agency — Automated Job Description Generator
Consultant: Marco Martins
Date: April 2026

What I Built

I implemented a complete LLM-as-judge evaluation pipeline in Python using the OpenAI API. The pipeline takes a set of structured client briefs, generates job descriptions using GPT-4o, and evaluates each output using a second GPT-4o call acting as a judge. Results are saved to a JSON file with per-case and aggregate metrics including score, criteria assessment, token usage, cost, and time.

The pipeline covers five test cases designed to probe the most critical failure modes for the agency use case: faithful brief conversion, inclusivity, sparse input handling, conflicting instructions, and legally sensitive client notes.

Key Findings

- 1. The generator needs explicit rules, not just instructions.** Telling the model "do not add anything not in the brief" is insufficient. GPT-4o consistently added soft skills ("passionate", "dynamic", "motivated self-starter"), boilerplate sections ("Why Join Us", "What We Offer"), and in TC04 even a Bachelor's degree requirement — none of which were in the brief. These are deeply ingrained JD writing conventions the model has learned from training data. The generator prompt must explicitly prohibit these additions with specific examples, mirroring the judge's criteria. Without this alignment, the generator and judge operate on different standards.
- 2. Judge strictness is a dial, not a switch.** The first version of the judge was too lenient — it scored TC01 and TC02 as 5/5 despite soft-skill hallucinations and gendered pronouns. After adding explicit criteria for inclusivity, legal safety, and stricter faithfulness rules, average scores dropped from 3.6 to 2.2. TC01 dropped from 5/5 to 2/5 because the judge now flags words like "dynamic" and "passionate" as hallucinations. This may be too strict — standard JD copywriting conventions are not the same as fabricated requirements. Calibration with reference examples per score level is essential before production use.
- 3. Rule-based checks catch what the judge misses.** In TC02, the first judge scored 5/5 but the regex check flagged "he" and "his" in the output. The improved judge caught these after inclusivity criteria were added — but this gap demonstrates that LLM

judges should never be the only verification layer. Rule-based checks are fast, cheap, and deterministic, and should run in parallel as a safety net for high-stakes criteria like gendered language and discriminatory requirements.

4. **TC05 is the highest-risk case in production.** The generator reproduced "native English speakers" and "young and energetic" from the client note in both runs. This is a compliance failure that would expose the agency to legal liability under EU employment law if published. The improved judge correctly identified these as legal failures and capped the score at 2. This finding alone justifies mandatory human review for any brief containing client notes or freeform instructions.
-

Recommended Next Steps

- **Fix the generator prompt** with explicit prohibitions matching the judge criteria (no soft skills, no boilerplate sections, no requirements not in the brief).
- **Calibrate the judge** with five reference examples (one per score level) to anchor the strictness threshold before production use.
- **Add a pre-processing step** that strips or flags legally sensitive content from client briefs before passing them to the generator.
- **Run a human baseline** — have senior HR consultants write JDs from the same briefs and score them with the same judge, to establish what a 5/5 output actually looks like in this domain.